

```
In [161... # Import the system functions and download the data from the Think stats URL

from os.path import basename, exists

# This function is used to download all the required files used for NSFG

def download(url):
    filename = basename(url)
    if not exists(filename):
        from urllib.request import urlretrieve

        local, _ = urlretrieve(url, filename)
        print("Downloaded " + local)

download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/thinkst
download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/thinkpl
download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/nsfg.py
download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/first.p

download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/2002Fem
download(
    "https://github.com/AllenDowney/ThinkStats2/raw/master/code/2002FemPreg.
)
```

```
In [163... #import all the required functions need to be used for this exercise

import numpy as np
import nsfg
import first
import thinkstats2
import thinkplot
```

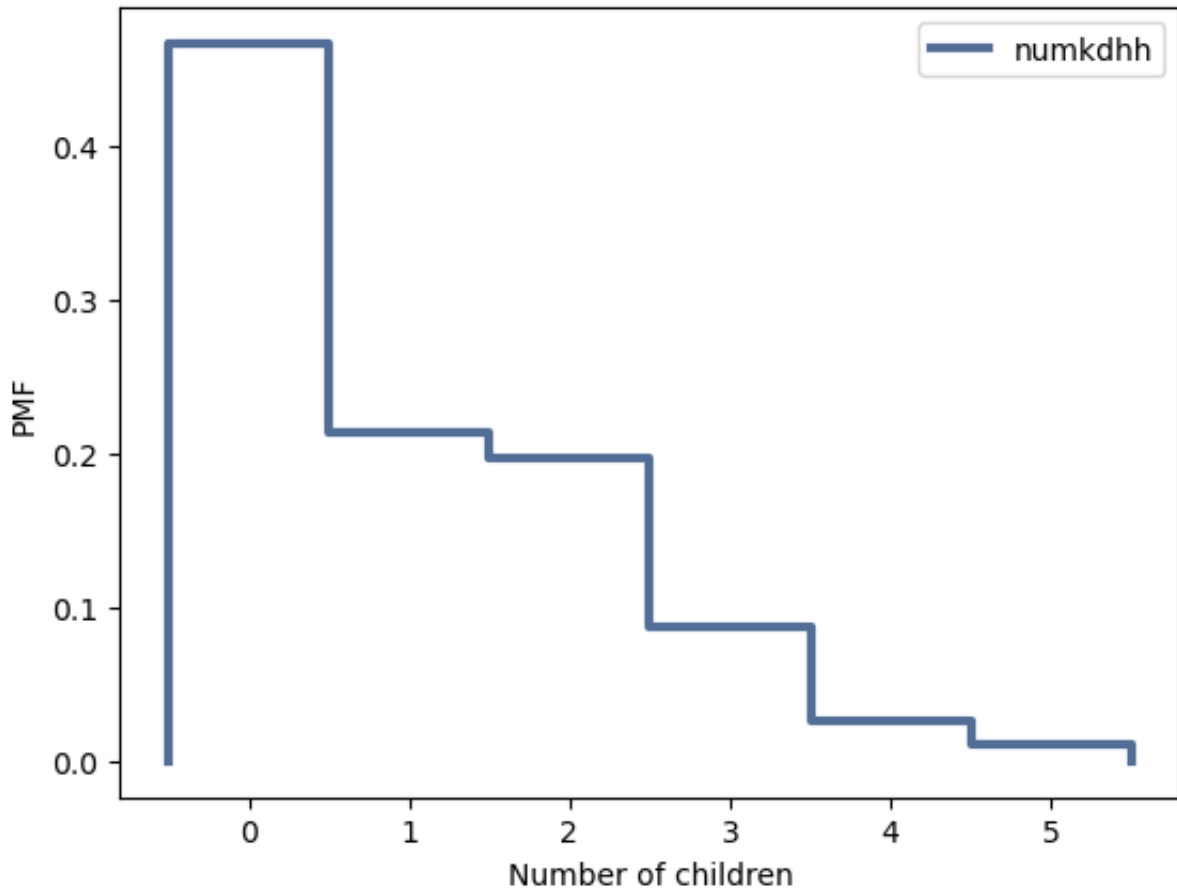
```
In [165... ***Exercise:** 3.1 && 3.2
```

```
In [167... # getting mean value using PMF
pmf.Mean()
```

```
Out[167... 0.507339536140008
```

```
In [169... resp = nsfg.ReadFemResp()
pmf = thinkstats2.Pmf(resp.numkdhh, label="numkdhh")

thinkplot.Pmf(pmf)
thinkplot.Config(xlabel="Number of children", ylabel="PMF")
```



In [170... *# This function is used to get the biased distributions for NSFG*

```
def BiasPmf(pmf, label):
    new_pmf = pmf.Copy(label=label)

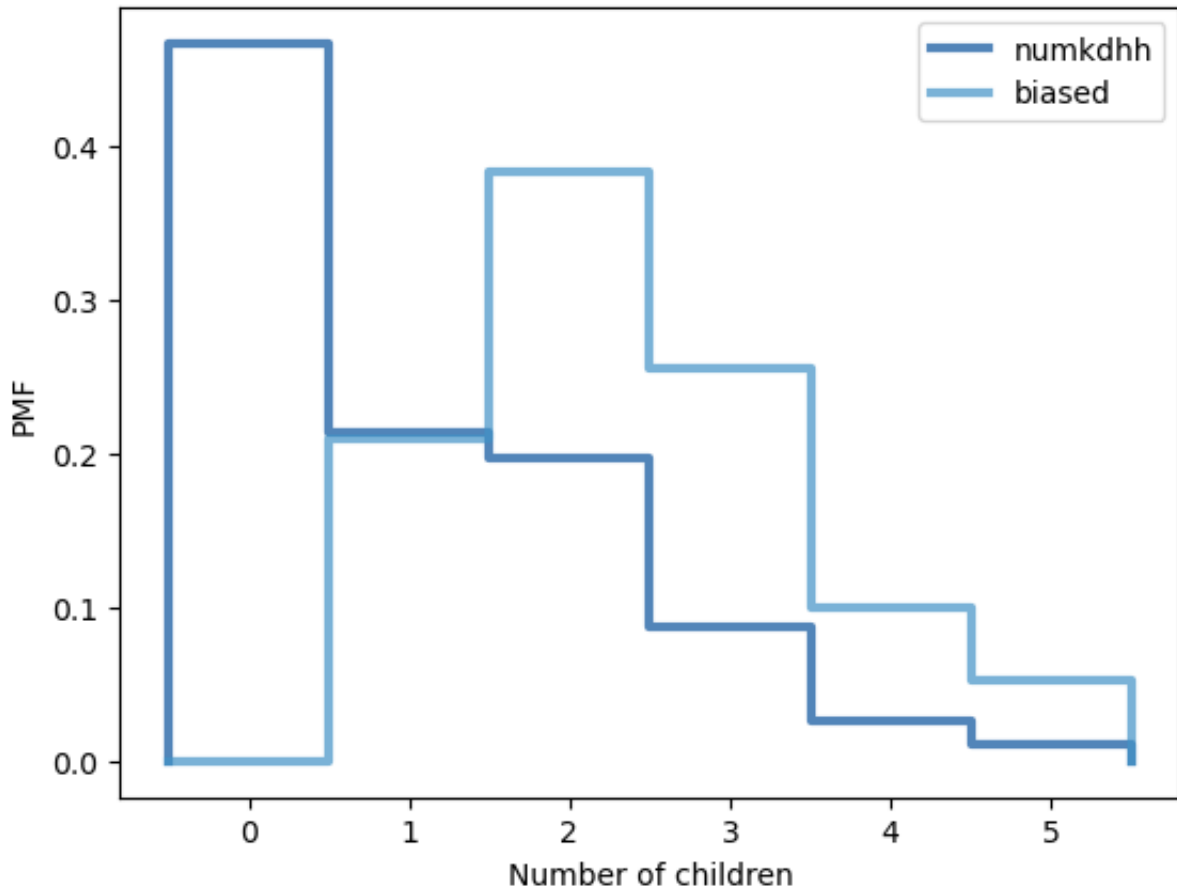
    for x, p in pmf.Items():
        new_pmf.Mult(x, x)

    new_pmf.Normalize()
    return new_pmf
```

In [171... `biased = BiasPmf(pmf, label="biased")`

In [172... *# Get graphical representation of numkdhh vs biased*

```
thinkplot.PrePlot(2)
thinkplot.Pmfs([pmf, biased])
thinkplot.Config(xlabel="Number of children", ylabel="PMF")
```



```
In [173... biased.Mean()
```

```
Out[173... 2.403679100664282
```

```
In [174... #Exercise 4.1 && 4.2
```

```
In [175... #`PercentileRank` computes the fraction of `scores` less than or equal to `y`
```

```
def PercentileRank(scores, your_score):
    count = 0
    for score in scores:
        if score <= your_score:
            count += 1

    percentile_rank = 100.0 * count / len(scores)
    return percentile_rank
```

```
In [176... def Percentile(scores, percentile_rank):
    scores.sort()
    for score in scores:
        if PercentileRank(scores, score) >= percentile_rank:
            return score
```

```
In [246... # function to evaluate CDF

def EvalCdf(sample, x):
    count = 0.0
    for value in sample:
        if value <= x:
            count += 1

    prob = count / len(sample)
    return prob
```

```
In [252... import first

live, firsts
```

Out[252...	(	caseid	pregordr	howpreg_n	howpreg_p	moscurrp	nowprgdk	pregend
1 \								
0		1	1	NaN	NaN	NaN	NaN	6.
0								
1		1	2	NaN	NaN	NaN	NaN	6.
0								
2		2	1	NaN	NaN	NaN	NaN	5.
0								
3		2	2	NaN	NaN	NaN	NaN	6.
0								
4		2	3	NaN	NaN	NaN	NaN	6.
0								
...		...	...	...	...	...	...	
...								
13581		12568	2	NaN	NaN	NaN	NaN	5.
0								
13584		12569	2	NaN	NaN	NaN	NaN	6.
0								
13588		12571	1	NaN	NaN	NaN	NaN	6.
0								
13591		12571	4	NaN	NaN	NaN	NaN	6.
0								
13592		12571	5	NaN	NaN	NaN	NaN	6.
0								
		pregend2	nbrnaliv	multbrth	...	laborfor_i	religion_i	metro_i
\								
0		NaN	1.0	NaN	...	0	0	0
1		NaN	1.0	NaN	...	0	0	0
2		NaN	3.0	5.0	...	0	0	0
3		NaN	1.0	NaN	...	0	0	0
4		NaN	1.0	NaN	...	0	0	0
...		...	...	...	...	...	...	...
13581		NaN	1.0	NaN	...	0	0	0

13584	NaN	1.0	NaN	...	0	0	0
13588	NaN	1.0	NaN	...	0	0	0
13591	NaN	1.0	NaN	...	0	0	0
13592	NaN	1.0	NaN	...	0	0	0

	basewgt	adj_mod_basewgt	finalwgt	secu_p	sest	cmintvw
\						
0	3410.389399	3869.349602	6448.271112	2	9	NaN
1	3410.389399	3869.349602	6448.271112	2	9	NaN
2	7226.301740	8567.549110	12999.542264	2	12	NaN
3	7226.301740	8567.549110	12999.542264	2	12	NaN
4	7226.301740	8567.549110	12999.542264	2	12	NaN
...	...	...	...	...	...	...
13581	2734.687353	4258.980140	7772.212858	2	28	NaN
13584	2580.967613	2925.167116	5075.164946	2	61	NaN
13588	4670.540953	5795.692880	6269.200989	1	78	NaN
13591	4670.540953	5795.692880	6269.200989	1	78	NaN
13592	4670.540953	5795.692880	6269.200989	1	78	NaN

	totalwgt_lb
0	8.8125
1	7.8750
2	9.1250
3	7.0000
4	6.1875
...	...
13581	6.3750
13584	6.3750
13588	6.1875
13591	7.5000
13592	7.5000

[9148 rows x 244 columns],

	caseid	pregordr	howpreg_n	howpreg_p	moscurrp	nowprgdk	pregend
1 \							
0	1	1	NaN	NaN	NaN	NaN	6.
0							
2	2	1	NaN	NaN	NaN	NaN	5.
0							
5	6	1	NaN	NaN	NaN	NaN	6.
0							
8	7	1	NaN	NaN	NaN	NaN	5.
0							
10	12	1	NaN	NaN	NaN	NaN	5.
0							
...	...	...	...	...	...	...	...
...							
13576	12565	1	NaN	NaN	NaN	NaN	6.
0							
13578	12566	1	NaN	NaN	NaN	NaN	6.

```

0
13581    12568        2      NaN      NaN      NaN      NaN      5.
0
13584    12569        2      NaN      NaN      NaN      NaN      6.
0
13588    12571        1      NaN      NaN      NaN      NaN      6.
0

```

```

      pregend2  nbrnaliv  multbrth  ...  laborfor_i  religion_i  metro_i
\
0          NaN      1.0      NaN  ...      0          0          0
2          NaN      3.0      5.0  ...      0          0          0
5          NaN      1.0      NaN  ...      0          0          0
8          NaN      1.0      NaN  ...      0          0          0
10         NaN      1.0      NaN  ...      0          0          0
...         ...      ...      ...  ...      ...      ...      ...
13576      NaN      1.0      NaN  ...      0          0          0
13578      NaN      1.0      NaN  ...      0          0          0
13581      NaN      1.0      NaN  ...      0          0          0
13584      NaN      1.0      NaN  ...      0          0          0
13588      NaN      1.0      NaN  ...      0          0          0

```

```

      basewgt  adj_mod_basewgt      finalwgt  secu_p  sest  cmintvw
\
0    3410.389399      3869.349602    6448.271112      2      9      NaN
2    7226.301740      8567.549110   12999.542264      2     12      NaN
5    4870.926435      5325.196999    8874.440799      1     23      NaN
8    3409.579565      3787.539000    6911.879921      2     14      NaN
10   3612.781968      4146.013572    6909.331618      1     31      NaN
...         ...      ...      ...      ...      ...      ...
13576  3195.641221      3834.241709    6652.409365      1     78      NaN
13578  2080.317155      2422.820274    2627.548587      2      2      NaN
13581  2734.687353      4258.980140    7772.212858      2     28      NaN
13584  2580.967613      2925.167116    5075.164946      2     61      NaN
13588  4670.540953      5795.692880    6269.200989      1     78      NaN

```

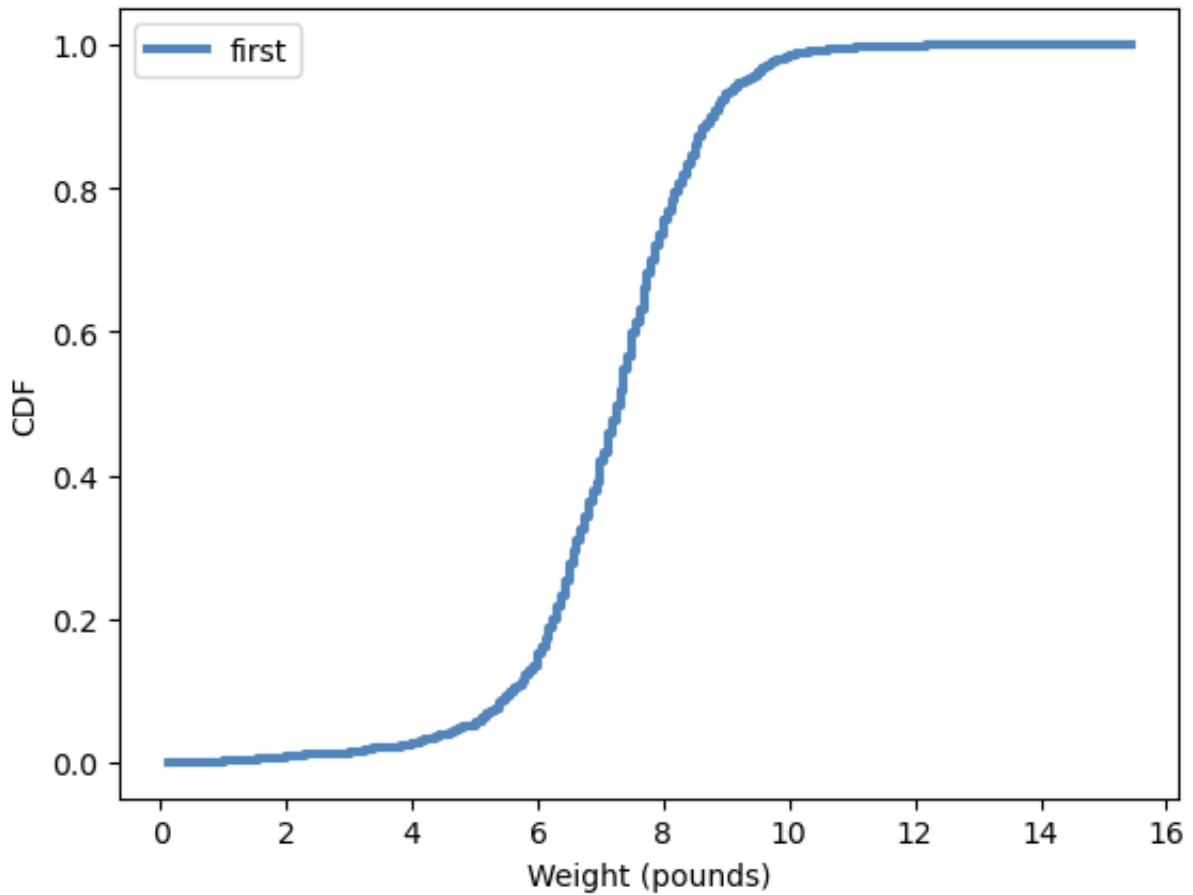
```

      totalwgt_lb
0          8.8125
2          9.1250
5          8.5625
8          7.5625
10         7.8125
...         ...
13576      6.4375
13578      6.0000
13581      6.3750
13584      6.3750
13588      6.1875

```

[4413 rows x 244 columns])

```
In [254... first_cdf = thinkstats2.Cdf(firsts.totalwgt_lb, label='first')  
  
thinkplot.PrePlot(2)  
thinkplot.Cdfs([first_cdf])  
thinkplot.Config(xlabel='Weight (pounds)', ylabel='CDF')
```



```
In [256... first_cdf.PercentileRank(8.5)
```

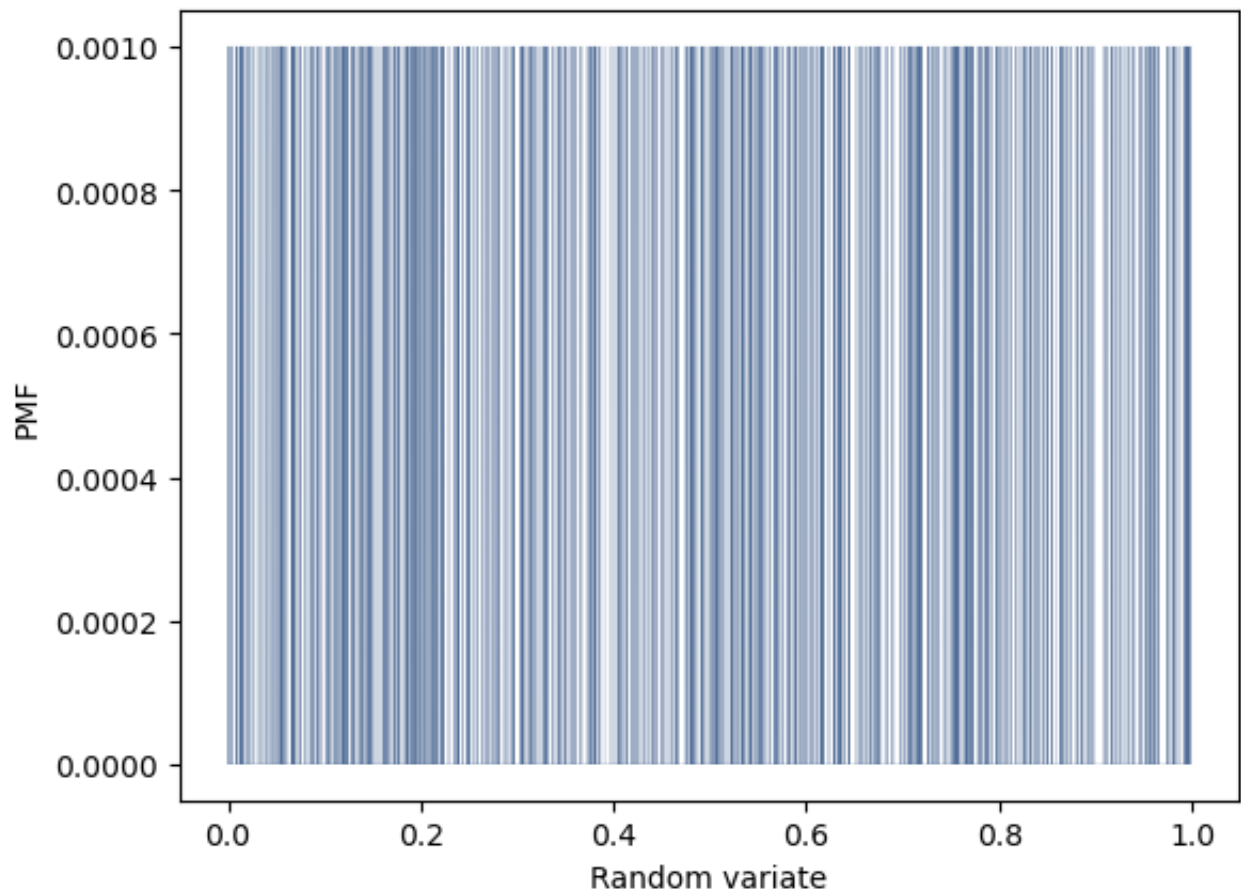
```
Out[256... 85.90419436167774
```

```
In [258... other_cdf.PercentileRank(8.5)
```

```
Out[258... 82.35294117647058
```

```
In [260... t = np.random.random(1000)
```

```
In [262... pmf = thinkstats2.Pmf(t)  
thinkplot.Pmf(pmf, linewidth=0.1)  
thinkplot.Config(xlabel='Random variate', ylabel='PMF')
```



```
In [264... cdf = thinkstats2.Cdf(t)
thinkplot.Cdf(cdf)
thinkplot.Config(xlabel='Random variate', ylabel='CDF')
```